

手写代码

1、手写Object.create

将传入的参数作为创建对象的原型

```
function create(obj) {  
  function f() {} // 定义一个空的构造函数 f  
  f.prototype = obj // 将 f 的原型指向传入的 obj 对象  
  return new f() // 通过 new 关键字创建一个继承 obj 的新对象  
}
```

2、手写new

```
function myNew(constructor, ...args) {  
  // 1. 创建一个新对象，并将其原型链接到构造函数的 prototype  
  const o = Object.create(constructor.prototype);  
  
  // 2. 在新对象 o 上执行构造函数，并传入参数  
  const result = constructor.apply(o, args);  
  
  // 3. 如果构造函数返回的是一个对象（object 或 function），则返回该对象  
  // 否则，返回新创建的对象 o  
  if (result && (typeof result === "object" || typeof result === "function")) {  
    return result;  
  } else {  
    return o;  
  }  
}  
  
// 示例：定义一个构造函数 a  
function a(name, age) {  
  this.name = name;  
  this.age = age;  
}  
  
console.log(myNew(a, "lina", 18))  
console.log(new a("lina", 18))  
console.log(myNew(Object))  
console.log(new Object)  
console.log(myNew(Function))  
console.log(new Function)
```

3、手写promise

```
class Commitment {  
  // 定义静态变量，表示 Promise 的三种状态  
  static PENDING = '待定'; // 初始状态，Promise 还未被处理  
  static FULFILLED = '成功'; // Promise 已成功解决  
  static REJECTED = '拒绝'; // Promise 已被拒绝
```

```

constructor(func) {
  // 初始状态为 PENDING
  this.status = Commitment.PENDING;
  this.result = null; // 存储 Promise 解决后的结果
  this.resolveCallbacks = []; // 存储成功回调函数
  this.rejectCallbacks = []; // 存储失败回调函数

  try {
    // 立即执行传入的函数 `func`，并传递 `resolve` 和 `reject`
    func(this.resolve.bind(this), this.reject.bind(this));
  } catch (error) {
    // 捕获 `func` 内部的错误，直接执行 `reject`
    this.reject(error);
  }
}

// 处理成功的逻辑
resolve(result) {
  setTimeout(() => {
    if (this.status === Commitment.PENDING) { // 只有 PENDING 状态才能转换
      this.status = Commitment.FULFILLED;
      this.result = result;
      // 依次执行存储的成功回调
      this.resolveCallbacks.forEach(callback => {
        callback(result);
      });
    }
  });
}

// 处理失败的逻辑
reject(result) {
  setTimeout(() => {
    if (this.status === Commitment.PENDING) { // 只有 PENDING 状态才能转换
      this.status = Commitment.REJECTED;
      this.result = result;
      // 依次执行存储的失败回调
      this.rejectCallbacks.forEach(callback => {
        callback(result);
      });
    }
  });
}

// `then` 方法用于注册成功和失败的回调
then(onFULFILLED, onREJECTED) {
  return new Commitment((resolve, reject) => {
    // 如果传入的回调不是函数，则提供默认函数
    onFULFILLED = typeof onFULFILLED === 'function' ? onFULFILLED : () => { };
    onREJECTED = typeof onREJECTED === 'function' ? onREJECTED : () => { };

    if (this.status === Commitment.PENDING) {
      // 处于 PENDING 状态时，将回调函数存入数组，等待状态改变后执行
      this.resolveCallbacks.push(onFULFILLED);
      this.rejectCallbacks.push(onREJECTED);
    }

    if (this.status === Commitment.FULFILLED) {

```

```

        // 立即执行成功回调（异步）
        setTimeout(() => {
            onFULFILLED(this.result);
        });
    }

    if (this.status === Commitment.REJECTED) {
        // 立即执行失败回调（异步）
        setTimeout(() => {
            onREJECTED(this.result);
        });
    }
    });
}
}

```

总结

- 该类模拟了 `Promise` 的基本功能，包括：
 - 状态管理（`PENDING` → `FULFILLED` / `REJECTED`）。
 - 存储 `then` 里的回调，状态变化时执行。
 - 通过 `setTimeout` 确保回调是异步执行的（符合 ES6 规范）。
- 但它缺少链式调用、`catch`、`finally`，可进一步完善。

4、手写eventbus

```

// 1、作用：实现非父子组件通信（传值），组件通信方案的一种
// 2、本质：空的Vue实例（发布订阅模式）
// 3、用法：
// 准备一个EventBus->本质就是一个空的Vue实例（提供核心方法）
// 监听（接受）->$on
// 派发（通知）->$emit
// 删除/解绑：$off

// 定义Observer类，作为事件总线，负责实现组件之间的通信
class Observer {
    constructor() {
        // 存储所有事件及其回调函数的对应关系，事件名为键，回调函数数组为值
        this.items = {}
    }

    // 监听、订阅事件
    // 第一个参数 eventName：事件名称，第二个参数 cb：事件回调函数
    $on(eventName, cb) {
        // 如果该事件没有被监听过，则初始化该事件的回调数组
        if (!this.items[eventName]) {
            this.items[eventName] = []
        }
        // 将回调函数添加到事件的回调数组中
        this.items[eventName].push(cb)
    }

    // 发布、通知、广播事件
    // 第一个参数 eventName：事件名称，后面的参数是传递给回调函数的参数

```

```

$emit(eventName, ...args) {
  // 如果没有该事件的监听者，则直接返回
  if (!this.items[eventName]) {
    return
  }
  // 遍历事件对应的所有回调函数，并依次调用它们，传递参数
  this.items[eventName].forEach(cb => {
    cb(...args)
  })
}

// 删除/解绑事件的所有回调函数
$off(eventName) {
  // 清空该事件对应的回调函数列表
  this.items[eventName] = []
}
}

```

解释：

1. **Observer 类**：这是实现发布订阅模式的核心类，它能够处理事件的注册（`$on`）、触发（`$emit`）和注销（`$off`）。
2. `$on`：用于监听指定事件，当事件被触发时，会执行回调函数。
3. `$emit`：用于触发某个事件，所有监听该事件的回调都会被依次执行，并传递参数。
4. `$off`：用于解绑指定事件的所有监听回调，清空该事件的回调数组。

通过这个方式，Vue组件可以通过事件总线（`observer` 类）进行通信，而无需直接建立父子关系。

5、手写promise.all

- 适用于 **所有任务都必须成功** 的情况，否则立即失败（**短路**）。
- 只返回成功结果数组，不包含失败的信息。
- **更快失败**，提高性能。

```

// 自定义的 Promise.all 实现，接受一个包含多个 Promise 或值的数组作为参数
function promiseAll(promises) {
  // 用于存储最终的结果
  let res = []
  // 计数器，用于记录已完成的 Promise 数量
  let counter = 0

  // 返回一个新的 Promise，只有当所有 Promise 都成功时才会 resolve
  return new Promise((resolve, reject) => {
    // 如果传入的参数不是数组，直接 reject 一个错误
    if (!Array.isArray(promises)) {
      return reject(new Error('传入的参数必须是数组!'))
    }

    // 遍历 promises 数组中的每个元素
    for (let i = 0; i < promises.length; i++) {
      // 通过 Promise.resolve 将每个元素转化为 Promise 对象（即使它本身已经是一个 Promise）
      Promise.resolve(promises[i])
        .then(value => {
          // 处理每个 Promise 的成功结果
          counter++ // 完成一个 Promise，计数器加 1

```

```

        res[i] = value    // 将成功的结果存入对应位置

        // 如果所有 Promise 都已完成（即计数器等于 promises 数组的长度）
        if (counter == promises.length) {
            resolve(res)    // 返回所有 Promise 的结果
        }
    })
    .catch(e => reject(e))    // 如果某个 Promise 出现错误，则直接 reject 错误
}
})
}

// 定义一个模拟的 Promise, p1 会在 2 秒后返回 'p1'
function p1() {
    return new Promise((resolve, reject) => {
        setTimeout(() => {
            resolve('p1')
        }, 2000)
    })
}

// 定义另一个 Promise, p2 会立即返回 'p2'
function p2() {
    return new Promise((resolve, reject) => {
        resolve('p2')
    })
}

// 调用 promiseAll 函数，传入一个包含字符串、Promise 和函数的数组
promiseAll(['a', 'b', p1(), p2(), 'c'])
    .then((result) => {
        // 输出最终的结果数组
        console.log(result)
    })
}

```

代码功能：

1. **promiseAll 函数**：这个函数是自定义的 `Promise.all` 实现，接收一个数组作为参数，该数组可以包含 Promise 对象或普通值。它将依次等待每个 Promise 完成，并返回一个新的 Promise，最终将所有结果以数组的形式返回。
 - 如果输入参数不是数组，`promiseAll` 会立即返回一个拒绝的 Promise，带有错误信息。
 - 通过遍历输入数组中的每个元素，使用 `Promise.resolve` 确保每个元素都是一个 Promise。
 - 每当一个 Promise 完成时，更新结果数组和计数器，直到所有 Promise 完成并返回最终结果。
2. **p1 和 p2**：这两个函数分别返回不同的 Promise。`p1` 在 2 秒后返回 'p1'，`p2` 则立刻返回 'p2'。
3. **promiseAll(['a', 'b', p1(), p2(), 'c'])**：调用 `promiseAll` 函数，传入一个混合了普通值和 Promise 的数组。最终，`promiseAll` 会等待所有 Promise 执行完毕，并输出最终的结果数组。

输出:

```
['a', 'b', 'p1', 'p2', 'c']
```

在输出中, 'a' 和 'b', 'c' 是直接值, p1 和 p2 是从相应的 Promise 获得的值。

```
> Promise.all(["a", "b"])
< ◀ Promise {<fulfilled>: Array(2)} i
  ▶ [[Prototype]]: Promise
    [[PromiseState]]: "fulfilled"
  ▶ [[PromiseResult]]: Array(2)
    0: "a"
    1: "b"
    length: 2
  ▶ [[Prototype]]: Array(0)
```

6、手写promise-allsettled

- 适用于 **所有任务都需要执行**, 无论成功或失败。
- 适合记录多个任务的最终结果 (**日志、批量操作**)。
- **不会短路**, 等待所有任务完成。

```
// 自定义的 Promise.allSettled 实现, 接收一个包含多个 Promise 或值的数组作为参数
function promiseSettled(promises) {
  // 判断输入是否为数组, 如果不是则抛出错误
  if (!Array.isArray(promises)) {
    throw Error("输入参数必须为数组!")
  }

  // 用于存储每个 Promise 的最终结果
  let res = []
  // 用于计数, 记录已处理的 Promise 数量
  let count = 0
  // 用于计数, 记录已 settled (无论是 fulfilled 还是 rejected) 完成的 Promise 数量
  let resolvedcount = 0

  // 返回一个新的 Promise, 当所有 Promise 都 settled (无论是 fulfilled 还是 rejected)
  // 时 resolve
  return new Promise((resolve, reject) => {
    // 遍历 promises 数组中的每个元素
    for (let i = 0; i < promises.length; i++) {
      count++ // 处理一个 Promise, 计数器加 1

      // 将每个元素转为 Promise 并处理
      Promise.resolve(promises[i])
        .then((value) => {
          // 如果该 Promise 成功, 记录它的 fulfilled 状态和返回值
          resolvedcount++
          res[i] = {
            status: 'fulfilled',
            value: value
          }
        })
    }

    // 所有 Promise 都 settled 后, 返回结果数组
    resolve(res)
  })
}
```

```

    }
  }, (reason) => {
    // 如果该 Promise 失败，记录它的 rejected 状态和拒绝原因
    resolvedcount++
    res[i] = {
      status: 'rejected',
      reason: reason
    }
  })
  .finally(() => {
    // 每个 Promise 完成时（无论成功还是失败），检查是否所有 Promise 都已 settled
    if (resolvedcount >= count) {
      resolve(res) // 如果所有 Promise 已完成，返回结果数组
    }
  })
}
})
}
}

```

代码功能：

1. `promiseSettled` 函数：这是自定义实现的 `Promise.allSettled`，它接收一个包含多个 Promise 或普通值的数组作为参数。该函数返回一个新的 Promise，当数组中的所有 Promise（包括成功或失败）都 settled（无论成功或失败）时，最终会返回一个数组，数组的每个元素是一个对象，包含 `status`（`fulfilled` 或 `rejected`）和对应的结果（`value` 或 `reason`）。
 - 如果输入不是数组，则抛出错误。
 - 遍历所有的 Promise，使用 `Promise.resolve` 来确保每个元素是一个 Promise。
 - 每个 Promise 的 `then` 方法和 `catch` 方法处理其成功和失败的状态，分别记录对应的状态和结果。
 - `finally` 方法确保无论 Promise 成功还是失败，都能在所有 Promise 都处理完后返回结果。
2. `res`：用于存储每个 Promise 的最终状态和结果。
3. `count`：用于记录 Promise 的总数量。
4. `resolvedcount`：用于记录已 settled（无论是成功或失败）的 Promise 数量。只有当这个计数器达到总数时，才会返回结果。

输出：

返回一个结果数组，其中每个元素是一个对象，格式如下：

```

[
  { status: 'fulfilled', value: '成功的值' },
  { status: 'rejected', reason: '失败的原因' },
  ...
]

```

示例：

```
const promises = [
  Promise.resolve('Success 1'),
  Promise.reject('Error 1'),
  Promise.resolve('Success 2')
];

promiseSettled(promises).then(result => {
  console.log(result);
});
```

输出:

```
[
  { status: 'fulfilled', value: 'Success 1' },
  { status: 'rejected', reason: 'Error 1' },
  { status: 'fulfilled', value: 'Success 2' }
]
```

```
▼ (3) [{...}, {...}, {...}] ⓘ
  ▶ 0: {status: 'fulfilled', value: 'Success 1'}
  ▶ 1: {status: 'rejected', reason: 'Error 1'}
  ▶ 2: {status: 'fulfilled', value: 'Success 2'}
    length: 3
  ▶ [[Prototype]]: Array(0)
```

通过这个方式，`promiseSettled` 可以处理多个 Promise，无论它们是成功还是失败，并返回所有 Promise 的最终结果。

7、手写防抖

```
/**
 * 防抖函数（debounce）
 * 该函数返回一个新的函数，该函数在一定时间（delay）内如果被多次调用，
 * 只会执行最后一次调用的回调（fn），可以用于减少高频事件的触发次数，例如窗口滚动、输入框输入等。
 *
 * @param {Function} fn - 需要防抖处理的函数
 * @param {number} delay - 多少毫秒后执行函数
 * @returns {Function} - 返回一个经过防抖处理的函数
 */
function debounce(fn, delay) {
  // 定义一个定时器变量
  let timer = null

  return function () {
    // 保存 `this` 指向，确保 `fn` 里的 `this` 仍然指向调用者
    let context = this
    // 将传入的参数存储到 `args` 数组中
    let args = [...arguments]

    // 如果定时器已存在，则清除之前的定时器，重新计时
    if (timer) {
      clearTimeout(timer)
      timer = null
    }
  }
}
```

```

    }

    // 重新设置一个新的定时器，在 `delay` 时间后执行 `fn`
    timer = setTimeout(() => {
        fn.apply(context, args) // 使用 `apply` 确保 `fn` 使用正确的 `this` 和参数
    }, delay)
}
}

```

使用示例

```

function handleResize() {
    console.log('窗口大小改变了')
}

// 创建一个防抖的窗口大小改变事件
window.addEventListener('resize', debounce(handleResize, 500))//context ===
window

```

工作原理

1. 初始化时，`debounce` 生成一个新函数，该函数内部维护一个 `timer` 变量。
2. 调用返回的新函数时：
 - 如果 `timer` 不为空，说明之前的定时器还未执行，则清除它。
 - 重新创建一个新的定时器，在 `delay` 毫秒后执行 `fn`。
3. 最终效果：
 - 只有在 `delay` 时间内没有再次调用该函数时，`fn` 才会执行。
 - 适用于输入框搜索优化、按钮点击防抖、窗口大小调整等场景。

8、手写节流

1. 时间戳写法（第一次立即执行）

```

/**
 * 节流函数（时间戳写法）
 * 第一次触发事件时立即执行，后续在 `delay` 时间间隔内不会重复执行。
 * 适用于希望立即执行，并且控制调用频率的场景，例如 `mousemove`、`scroll` 等高频事件。
 *
 * @param {Function} fn - 需要节流的函数
 * @param {number} delay - 执行函数的最小时间间隔（毫秒）
 * @returns {Function} - 返回一个节流处理后的函数
 */
function throttle(fn, delay) {
    // 记录上次执行函数的时间
    let last = new Date()//last 初始化时存储的是 throttle 创建的时间

    return function () {
        // 记录当前时间
        let now = new Date()
        // 保存 `this` 指向，确保 `fn` 里的 `this` 仍然指向调用者
        let context = this
        // 获取传入的参数
        let args = [...arguments]
    }
}

```

```
// 只有当距离上次执行的时间超过 `delay` 毫秒时，才会执行 `fn`
if (now - last >= delay) {
  last = now // 更新上次执行时间
  fn.apply(context, args) // 执行函数
}
}
```

时间戳写法特点

- 第一次触发时立即执行，不会等待 `delay` 时间。
- 间隔 `delay` 毫秒后，下一次执行才能触发。
- 适用于需要立即响应用户操作的场景，例如 `mousemove`、`scroll` 事件监听。

2. 定时器写法（第一次延时执行）

```
/**
 * 节流函数（定时器写法）
 * 第一次触发事件不会立即执行，而是等 `delay` 毫秒后才执行，之后每 `delay` 毫秒最多执行一次。
 * 适用于希望减少触发频率，但又不影响最终执行结果的场景，例如 **搜索输入**、**按钮点击防抖**。
 */
* @param {Function} fn - 需要节流的函数
* @param {number} delay - 执行函数的最小时间间隔（毫秒）
* @returns {Function} - 返回一个节流处理后的函数
*/
function throttle1(fn, delay) {
  // 记录定时器
  let timer = null

  return function () {
    // 保存 `this` 指向
    let context = this
    // 获取传入的参数
    let args = [...arguments]

    // 只有当 `timer` 为空时，才会启动新的定时器
    if (!timer) {
      timer = setTimeout(() => {
        fn.apply(context, args) // 执行函数
        clearTimeout(timer) // 清除定时器
        timer = null // 释放 `timer`，下次才会继续触发
      }, delay)
    }
  }
}
```

定时器写法特点

- 第一次触发不会立即执行，而是等待 `delay` 毫秒后才执行。
- 在 `delay` 时间间隔内，只能执行一次，即使期间多次触发。
- 适用于 按钮点击、表单输入事件，防止多次提交请求。

3. 使用示例

```
function logMessage() {
  console.log('触发了', new Date().toLocaleTimeString())
}

// 时间戳节流：立即执行
window.addEventListener('scroll', throttle(logMessage, 2000))

// 定时器节流：延迟执行
window.addEventListener('scroll', throttle1(logMessage, 2000))
```

- `throttle(logMessage, 2000)`：第一次立即执行，之后每隔 2 秒执行一次。
- `throttle1(logMessage, 2000)`：第一次不会立即执行，2 秒后执行，之后也是每 2 秒执行一次。

4. 时间戳 vs. 定时器 对比

对比项	时间戳写法	定时器写法
第一次触发	立即执行	延迟执行
后续执行间隔	<code>delay</code> 毫秒后才能再次触发	<code>delay</code> 毫秒后执行一次
适用场景	<code>mousemove</code> 、 <code>scroll</code> ，希望立即执行的场景	<code>input</code> 、 <code>click</code> ，防止多次请求提交

如果希望 第一次立即执行，用 时间戳写法；如果希望 第一次延迟执行，用 定时器写法。

9、手写apply、call

作用：改变函数执行时this的指向

特点：

- 1、添加在函数原型上的方法 `Function.prototype`
- 2、调了call和apply 函数会立即执行

call的特点：

- 1、当第一个参数传递的是undefined或null，内部会让this指向window
- 2、当第一个参数传递的是基本类型，内部会调用相应的构造函数创建对应的实例，让this指向这个对象
- 3、当第一个参数传递的是引用类型（比如对象、数组），内部this指向这个对象

```
// 定义一个测试函数
function fn(x, y) {
  console.log(this.name + x + y); // this 指向调用者
}

// 定义一个对象，后续用作 `this` 绑定的上下文
var obj = {
  name: "haoyue",
```

```

};

// ===== 重写 call 方法 =====
Function.prototype._call = function (ctx, ...args) {
  // 1. 处理 `ctx` 为空 (`null` 或 `undefined`) 的情况, 默认为 `window` (或全局对象)
  const o = ctx === undefined || ctx === null ? window : Object(ctx);

  // 2. 在 `o` (目标对象) 上创建一个临时方法, 绑定当前函数 (即 `this`)
  // console.log("this", this);
  // console.log("o1", o);
  o.fn = this;
  // console.log("o2", o);
  // 3. 调用 `fn` 方法, 并传递参数
  const result = o.fn(...args);

  // 4. 删除临时方法, 避免污染目标对象
  delete o.fn;
  console.log("o3", o);
  // 5. 返回函数执行结果 (如果有返回值)
  return result;
};

// 测试 `_call` 方法, 等价于 `fn.call(obj, 1, 2)`
fn._call(obj, 1, 2); // 输出: haoyue12

// ===== 重写 apply 方法 =====
Function.prototype._apply = function (ctx, array = []) {
  // 1. 处理 `ctx` 为空 (`null` 或 `undefined`) 的情况, 默认为 `window` (或全局对象)
  const o = (ctx === undefined || ctx === null) ? window : Object(ctx);

  // 2. 在 `o` (目标对象) 上创建一个临时方法, 绑定当前函数 (即 `this`)
  o.a = this;

  // 3. 调用 `a` 方法, 并传递参数数组
  const result = o.a(...array);

  // 4. 删除临时方法, 避免污染目标对象
  delete o.a;

  // 5. 返回函数执行结果 (如果有返回值)
  return result;
};

// 测试 `_apply` 方法, 等价于 `fn.apply(obj, [3, 4])`
fn._apply(obj, [3, 4]); // 输出: haoyue34

```

```

this f fn(x, y) {
  console.log(this.name + x + y); // this 指向调用者
}

```

```

o1 ▶ {name: 'haoyue'}

```

```

o2 ▶ {name: 'haoyue', fn: f}

```

```

haoyue12

```

```

o3 ▶ {name: 'haoyue'}

```

为什么 `console.log("this", this);` 输出的是 `fn` 本身?

- ✓ 在 `Function.prototype._call` 内部, `this` 指向的是调用 `_call` 的函数, 即 `fn`。
- ✓ 这是因为 `_call` 是定义在 `Function.prototype` 上的, 并通过 `fn._call(obj, 1, 2)` 调用。
- ✓ 通过 `call` 或 `apply` 绑定 `this` 后, `this` 的指向取决于调用 `_call` 的函数, 即 `fn`。

`call` 与 `apply` 的区别

- `call(thisArg, arg1, arg2, ...)`
 - 直接传递多个参数。
 - `_call` 方法使用 **剩余参数** `...args` 来接收参数并传递给 `fn`。
- `apply(thisArg, [argsArray])`
 - 传递 **数组** 作为参数。
 - `_apply` 方法确保 `array` 是一个数组, 使用 **展开运算符** `...array` 来传递给 `fn`。

`call` & `apply` 实现思路

1. 确保 `this` 绑定的对象 (`ctx`) 存在, 如果是 `null` 或 `undefined`, 默认绑定 `window` (浏览器环境) 或 `global` (Node.js 环境)。
2. 将当前函数赋值给目标对象的临时属性, 通过 `o.fn = this` 实现函数的 `this` 绑定。
3. 执行该函数, 并传递参数:
 - `call` 直接展开参数列表 `fn(...args)`
 - `apply` 通过数组展开 `fn(...array)`
4. 删除临时属性, 防止污染 `ctx`。

`call` & `apply` 执行流程

1 `_call` 执行

```
fn._call(obj, 1, 2);
```

等价于:

```
obj.fn = fn;  
obj.fn(1, 2); // 执行  
delete obj.fn; // 清理
```

输出:

```
haoyue12
```

2 `_apply` 执行

```
fn._apply(obj, [3, 4]);
```

等价于：

```
obj.a = fn;  
obj.a(3, 4); // 执行  
delete obj.a; // 清理
```

输出：

```
haoyue34
```

🔗 总结

- ✓ `_call` 和 `_apply` 都是基于 在目标对象上创建临时方法 实现 `this` 绑定。
- ✓ `_call` 适用于 直接传递多个参数，而 `_apply` 适用于 传递数组参数。
- ✓ 调用后会 删除临时方法，避免污染原始对象。

10、手写bind

作用： 返回一个内部改变了this指向的新的函数（引用）

特点：

- 1、添加在函数原型上的方法 `Function.prototype`
- 2、调了bind函数不会立即执行

```
// ===== 自定义 _call 方法 =====  
Function.prototype._call = function (ctx, ...args) {  
  const o = (ctx === undefined || ctx === null) ? window : Object(ctx);  
  o.fn = this;  
  const result = o.fn(...args);  
  delete o.fn;  
  return result;  
};  
  
// ===== 自定义 _apply 方法 =====  
Function.prototype._apply = function (ctx, array = []) {  
  const o = (ctx === undefined || ctx === null) ? window : Object(ctx);  
  o.a = this;  
  const result = o.a(...array);  
  return result;  
};  
  
// ===== 自定义 _bind 方法 =====  
Function.prototype._bind = function (ctx, ...args) {  
  // 1. 记录当前函数（`this` 指向调用 `_bind` 的函数）  
  const _self = this;  
  
  // 2. 返回一个新的函数，延迟执行  
  return function (...rest) {
```

```

// 3. 调用 `_call` 方法, 将 `ctx` 作为 `this`, 并合并 `args` 和 `rest` 作为最终参数
return _self._call(ctx, ...args, ...rest);
// 也可以使用 `_apply` 实现同样的功能
// return _self._apply(ctx, [...args, ...rest]);
};
};

function greet(greeting, punctuation) {
  console.log(this.name + " says: " + greeting + punctuation);
}

const person = { name: "Alice" };

// 绑定 `this` 并预设参数
const boundGreet = greet._bind(person, "Hello");

// 调用返回的新函数
boundGreet("!"); // 相当于 greet.call(person, "Hello", "!")

```

```

Function.prototype._call = function (ctx, ...args) {
  const o = ctx === undefined || ctx === null ? window : Object(ctx);
  o.fn = this;
  const result = o.fn(...args);
  delete o.fn;
  return result;
};
fn._call(obj, 1, 2);

```

指向实际的函数fn

方法名	作用	传参方式	立即执行	返回值
<code>_call</code>	改变 <code>this</code> 指向, 并执行函数	直接传递参数	☑ 立即执行	返回函数执行结果
<code>_apply</code>	改变 <code>this</code> 指向, 并执行函数	传递参数数组	☑ 立即执行	返回函数执行结果
<code>_bind</code>	绑定 <code>this</code> , 但不会立即执行, 返回新函数	直接传递参数	✗ 不会立即执行	返回新函数, 需手动调用

11、promise失败重试

```

const rate = 0.9;
function request() {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      //随机数计算 来实现失败与成功
      let res = Math.random(); //0-0.9999之间
      if (res > rate) {
        resolve(res);
      } else {
        reject(res);
      }
    });
  });
};

```

```

}
function retry(func, times, delay) {
  return new Promise((resolve, reject) => {
    //func是一件事，将它封装起来，才能times--时调用
    let inner = async function inner() {
      try {
        const result = await func();
        resolve(result);
      } catch (e) {
        if (times > 0) {
          console.log(`开始重试，剩余次数: ${times - 1}`);
          times--; // 先打印后递减，逻辑更清晰

          // 延迟执行
          setTimeout(() => {
            inner(); // 再次尝试
          }, delay);
        } else {
          // 没有剩余的重试次数，彻底失败
          reject(e);
        }
      }
    };
    inner();
  });
}
//函数设计
retry(request, 3, 2000)
  .then((res) => {
    console.log(res, "成功");
  })
  .catch((e) => {
    console.log(e, "失败");
  });

```

12、js对象转化为树形结构

```

function arrTotree(arr) {
  // 1. 过滤出所有父节点（pid 为 0 的节点）
  let parent = arr.filter((p) => p.pid == 0);

  // 2. 过滤出所有子节点（pid 不为 0 的节点）
  let children = arr.filter((c) => c.pid != 0);

  // 3. 调用递归函数，将父节点和子节点关联起来
  dfs(parent, children);

  // 4. 输出最终的树形结构
  console.log(parent);

  // 递归函数：为父节点添加子节点
  function dfs(parent, children) {
    parent.map((p) => { // 遍历每个父节点
      children.map((c, i) => { // 遍历每个子节点
        if (p.id == c.pid) { // 当前子节点 c 是 p 的子节点
          let _children = JSON.parse(JSON.stringify(children)); // 深拷贝
          children

```

```

        _children.splice(i, 1);    // 删除当前节点 c，避免重复处理

        // 递归处理 c 的子节点
        dfs([c], _children);

        // 如果父节点已经有子节点，则将当前子节点添加到父节点的 `children` 数组中
        if (p.children) {
            p.children.push(c);
        } else {
            // 如果父节点没有 `children` 属性，则初始化为一个数组并将子节点添加进去
            p.children = [c];
        }
    }
    });
    });
    return parent;    // 返回构建好的父节点数组
}
}

```

示例：调用 arrTotree 函数

```

const arr = [
  { id: 1, pid: 0, name: '根节点' },
  { id: 2, pid: 1, name: '子节点1' },
  { id: 3, pid: 1, name: '子节点2' },
  { id: 4, pid: 2, name: '子节点1-1' },
  { id: 5, pid: 3, name: '子节点2-1' }
];

```

调用 arrTotree 函数：

```
arrTotree(arr);
```

最终的树形结构

在执行 arrTotree(arr) 后，最终得到的树形结构如下：

```

[
  {
    id: 1,
    pid: 0,
    name: "根节点",
    children: [
      {
        id: 2,
        pid: 1,
        name: "子节点1",
        children: [
          {
            id: 4,
            pid: 2,
            name: "子节点1-1"
          }
        ]
      }
    ]
  },

```

```

    {
      id: 3,
      pid: 1,
      name: "子节点2",
      children: [
        {
          id: 5,
          pid: 3,
          name: "子节点2-1"
        }
      ]
    }
  ]
}
]

```

树形结构图

```

根节点
|
├── 子节点1
|   └── 子节点1-1
|
└── 子节点2
    └── 子节点2-1

```

结果分析

1. 根节点 是树的根节点，没有父节点 (pid: 0)。
2. 子节点1 和 子节点2 是 根节点 的子节点，分别对应 pid: 1。
3. 子节点1-1 是 子节点1 的子节点，子节点2-1 是 子节点2 的子节点。
4. 每个节点的 children 数组记录了它的子节点，最终形成了树状结构。

13、手写instanceof

```

function myinstanceof (left, right) {
  // 获取 `left` 的原型对象
  let proto = Object.getPrototypeOf(left); // 有些时候等价于 __proto__
  // 获取 `right` 的原型 (构造函数的 `prototype`)
  let prototype = right.prototype;

  // 循环查找 `left` 的原型链
  while (true) {
    if (!proto) return false; // 如果 `proto` 为空，则 `left` 不是 `right` 的实例
    if (proto === prototype) return true; // 如果找到了 `right.prototype`，返回 `true`

    // 沿着原型链继续向上查找
    proto = Object.getPrototypeOf(proto);
  }
}

```

执行逻辑

该函数模拟 `instanceof` 关键字的行为，检查 `left`（某个对象）是否在 `right`（构造函数）的原型链上。

即：`left` 的原型链上是否能找到 `right.prototype`。

工作机制

1. 获取 `left` 的原型对象

- 通过 `Object.getPrototypeOf(left)` 获取 `left` 的 `__proto__`，赋值给 `proto`。

2. 获取 `right` 的 `prototype`

- `right.prototype` 是构造函数 `right` 创建的实例应当继承的原型。

3. 遍历 `left` 的原型链

- 进入 `while(true)` 无限循环：
 - 若 `proto` 为 `null`，说明 `left` 已经遍历到 `Object.prototype.__proto__`，仍未找到匹配的 `right.prototype`，返回 `false`。
 - 若 `proto === prototype`，说明 `left` 是 `right` 的实例，返回 `true`。
 - 否则，不断向上查找 `left` 的原型链。
-

示例

```
function Person() {}
const p = new Person();

console.log(myinstanceof(p, Person)); // true
console.log(myinstanceof(p, Object)); // true
console.log(myinstanceof(p, Array)); // false
console.log(myinstanceof([], Array)); // true
console.log(myinstanceof([], Object)); // true
console.log(myinstanceof(123, Number)); // false （基本类型没有原型链）
```

`instanceof` 原理

在 JavaScript 中：

```
p instanceof Person;
```

等价于：

```
Person.prototype 是否出现在 p 的 __proto__ 原型链上。
```

本质上是沿着 `p.__proto__` 不断向上查找，是否能找到 `Person.prototype`。

特殊情况

```
console.log(myinstanceof(null, Object)); // false
console.log(myinstanceof(undefined, Object)); // false
```

- `null` 和 `undefined` 没有原型链，因此直接返回 `false`。

总结

- **核心逻辑**：沿着 `left` 的原型链向上查找 `right.prototype`。
- **终止条件**：
 1. `proto === null`，返回 `false`（遍历到顶）。
 2. `proto === prototype`，返回 `true`（匹配成功）。
- **注意点**：
 - `null` 和 `undefined` 没有原型链，直接 `false`。
 - `instanceof` 只能用于引用类型，基本数据类型需要使用 `typeof` 进行类型判断。

14、用promise封装ajax

```
function getJson(url) {
  // 返回一个Promise，异步获取JSON数据
  return new Promise((resolve, reject) => {

    // 创建 XMLHttpRequest 对象
    let xhr = new XMLHttpRequest();

    // 配置请求类型、请求URL，设置异步为 true
    xhr.open("GET", url, true);

    // 当请求状态发生变化时触发该函数
    xhr.onreadystatechange = function() {
      // 检查请求是否已完成（请求完成readyState 为 4）
      if (xhr.readyState !== 4) return;

      // 检查HTTP状态是否为200（即请求成功）
      if (xhr.status == 200) {
        // 请求成功，解析响应并传递给调用者
        resolve(xhr.response);
      } else {
        // 请求失败，抛出一个错误，包含错误信息
        reject(new Error(xhr.responseText));
      }
    };

    // 当请求发生网络错误时触发该函数
    xhr.onerror = function() {
      // 发生网络错误时，抛出一个错误，包含错误的状态文本
      reject(new Error(xhr.statusText));
    };

    // 设置返回的响应类型为 JSON
    xhr.responseType = "json";
  });
}
```

```
// 设置请求头，指明期望返回 JSON 格式的数据
xhr.setRequestHeader('Accept', 'application/json');

// 发送请求，不带请求体
xhr.send(null);
});
}
```

执行流程：

1. **创建 XMLHttpRequest 实例**：通过 `new XMLHttpRequest()` 创建一个新的 XMLHttpRequest 对象 `xhr`，用于发起 HTTP 请求。
2. **配置请求**：使用 `xhr.open("GET", url, true)` 配置一个异步的 GET 请求，指定 URL 和请求类型。
3. **监听请求状态**：
 - `onreadystatechange` 事件监听请求状态变化，`xhr.readyState` 等于 4 表示请求完成。
 - 如果状态码 `xhr.status` 为 200，表示请求成功，调用 `resolve` 将 `xhr.response`（即响应的数据）作为结果传递出去。
 - 如果请求失败，调用 `reject` 抛出一个错误对象。
4. **网络错误处理**：通过 `onerror` 事件监听网络请求错误，发生错误时调用 `reject`，抛出带有 `xhr.statusText` 的错误信息。
5. **响应类型和请求头设置**：
 - 设置响应类型 `xhr.responseType = "json"`，表示期望服务器返回 JSON 数据。
 - 设置请求头 `xhr.setRequestHeader('Accept', 'application/json')`，告诉服务器客户端希望接收 JSON 格式的响应。
6. **发送请求**：调用 `xhr.send(null)` 发送请求，传递 `null` 表示没有请求体。

优缺点分析：

- 优点：
 - 使用 `XMLHttpRequest` 进行传统的异步请求处理，支持大多数浏览器。
 - 可以处理 JSON 格式的响应数据，简洁高效。
- 缺点：
 - 在现代 JavaScript 开发中，通常推荐使用 `fetch` API，它提供了更简洁、更灵活的方式来处理 HTTP 请求和响应。

15、promise并发数限制

```
// 定义并发请求函数
function concurRequest(urls, maxNum) {
  return new Promise(resolve => {
    // 如果 URL 数组为空，直接返回空数组
    if (urls.length === 0) {
      resolve([]);
      return;
    }

    const result = []; // 用于存储每个请求的结果（成功或失败）
    let index = 0; // 下一个要请求的 URL 在数组中的索引
    let count = 0; // 记录已完成的请求数量
```

```

// 发送请求的函数
async function request() {
  // 如果所有的请求都已经发送完，则直接返回
  if (index === urls.length) return;

  const i = index; // 记录当前请求的索引
  const url = urls[index]; // 获取当前要请求的 URL
  index++; // 更新索引，准备下一个请求

  try {
    const res = await fetch(url); // 发送 HTTP 请求
    result[i] = res; // 请求成功，将响应存入结果数组
  } catch (error) {
    result[i] = error; // 请求失败，将错误存入结果数组
  } finally {
    count++; // 更新已完成的请求数

    // 当所有请求都完成时，打印 "over"
    if (count === urls.length) {
      console.log("over");
      resolve(result); // 解析 Promise，返回最终结果
    }

    // 继续发送下一个请求，保持最大并发数
    request();
  }

  console.log(result); // 打印当前已完成的请求结果
}

// 计算初始需要发送的请求数量（最多 maxNum 个并发）
const times = Math.min(maxNum, urls.length);
for (let i = 0; i < times; i++) {
  request(); // 启动初始的 maxNum 个请求
}
});
}

// 生成 100 个测试 URL
const urls = [];
for (let i = 0; i <= 100; i++) {
  urls.push(`https://jsonplaceholder.typicode.com/todos/${i}`);
}

// 调用函数并发执行 20 个请求
concurRequest(urls, 20).then(res => {
  console.log(res); // 打印最终的所有请求结果
});

```

16、手写迭代器

for of 是作为 ES6 新增的遍历方式，允许遍历一个含有 iterator 接口的数据结构（数组、对象等）并且返回各项的值，普通的对象用 for...of 遍历是会报错的

如果不是类数组对象，就给对象添加一个 [Symbol.iterator] 属性，并指向一个迭代器即可。

可迭代对象 (Iterable Object) 指的是可以使用 `for...of` 循环遍历的对象，或者可以用 `...` 扩展运算符展开的对象。常见的可迭代对象包括：

- 数组 (Array)
- 字符串 (String)
- Set
- Map
- arguments (函数参数对象)
- NodeList (DOM 查询结果)
- 自定义对象 (实现 `Symbol.iterator` 方法)

判断对象是否可迭代

一个对象是否可迭代，取决于它是否实现了 `Symbol.iterator` 方法：

```
let arr = [1, 2, 3];
console.log(typeof arr[Symbol.iterator]); // function, 说明数组是可迭代对象

let obj = { a: 1, b: 2 };
console.log(typeof obj[Symbol.iterator]); // undefined, 普通对象默认不可迭代
```

迭代协议 (Iteration Protocols)

可迭代对象必须符合**迭代协议**，即包含 `Symbol.iterator` 方法，它返回一个**迭代器对象** (Iterator)。**迭代器对象**必须具有 `next()` 方法，每次调用返回：

```
{ value: 值, done: 是否迭代完成 }
```

- `value`：当前迭代出的值
- `done`：是否迭代完成，`true` 表示结束，`false` 表示可以继续

```
// 定义一个对象 obj
var obj = {
  a: 1,
  b: 2,
  c: 3
};

// 给对象 obj 添加 [Symbol.iterator] 方法，使其变成可迭代对象
obj[Symbol.iterator] = function () {
  let keys = Object.keys(this); // 获取对象的所有键名 (返回 ["a", "b", "c"])
  let count = 0; // 记录当前迭代的索引
  let self = this; // 避免直接访问 obj，确保作用域正确

  return {
    next() {
      if (count < keys.length) {
        // 继续迭代，返回当前 key 对应的 value，并将索引加 1
        return { value: self[keys[count++]], done: false };
      } else {
        // 迭代结束，返回 done: true
        return { value: undefined, done: true };
      }
    }
  };
};
```

```
};

// 使用 for...of 遍历 obj（因为 obj 现在是一个可迭代对象）
for (let k of obj) {
  console.log(k); // 依次输出 1, 2, 3
}
```

17、第 1 秒后输出 1，第 2 秒后输出 2，第 3 秒后输出 3...，直到 n 秒

1. 使用 Promise 实现

```
function fn(n) {
  let count = 1; // 初始化计数器

  function sleep() {
    if (count === n + 1) {
      return; // 递归终止条件
    }

    new Promise((resolve, reject) => {
      setTimeout(() => {
        console.log(count); // 输出当前计数
        count++; // 计数 +1
        resolve(); // 任务完成，触发 then
      }, count * 1000); // 延迟 `count` 秒
    }).then(sleep); // 递归调用 sleep 以继续下一个计时
  }

  sleep(); // 立即调用 sleep 开始执行
}

fn(10);
```

执行流程

1. `sleep()` 递归调用自身，每次 `setTimeout` 的延迟时间是 `count * 1000`（即 1s, 2s, 3s...）。
2. `new Promise()` 创建一个异步任务，`setTimeout` 触发后 `resolve()`，然后 `then(sleep)` 继续递归调用 `sleep()`。
3. 直到 `count > n` 时停止递归。

存在的问题

- `setTimeout` 的触发时间是基于 `count * 1000`，但 `sleep()` 在 `then()` 之后不会等待上一个 `setTimeout` 完成，会导致：
 - 第 1 秒输出 1
 - 第 2 秒输出 2
 - 第 3 秒输出 3...
 - 但时间可能会不太精准（因为 `setTimeout` 是异步执行）。

2. 使用 `async/await` 实现(推荐)

```
// 定义 sleep 函数, 返回一个 Promise, 在指定时间后 resolve
function sleep(ms) {
  return new Promise(resolve => setTimeout(resolve, ms));
}

// 定义 fn 函数
async function fn(n) {
  let count = 1;

  async function sleepAndIncrement() {
    while (count <= n) { // 直到 count > n 结束循环
      await sleep(count * 1000); // 等待 count * 1000 毫秒
      console.log(count); // 输出当前 count
      count++; // 递增
    }
  }

  await sleepAndIncrement(); // 调用并等待完成
}

fn(10);
```

执行流程

1. `sleep(ms)` 创建一个 `Promise`, 延迟 `ms` 毫秒后 `resolve()`。
2. `sleepAndIncrement()` 使用 `while` 循环, 每次 `await sleep(count * 1000)` 等待 `count` 秒, 然后输出 `count` 并递增。
3. `await sleepAndIncrement()` 确保 `fn(n)` 在全部执行完毕前不会退出。

优点

- ✓ `async/await` 让代码更清晰, 不需要递归
- ✓ `await sleep(count * 1000)` 真正等待 上一个 `setTimeout` 结束后再执行下一个

执行对比

版本	主要实现方式	代码可读性	时间控制
<code>Promise.then()</code>	递归调用 <code>then()</code>	一般	可能不精确 (多个 <code>setTimeout</code> 同时运行)
<code>async/await</code>	<code>await sleep(ms)</code> 循环	✓ 简洁 清晰	✓ 时间精准

18、手写发布者-订阅者模式

```
// 定义一个订阅者管理类 (Dep), 用于管理所有的观察者 (订阅者)
class Dep {
  constructor() {
    this.subarr = [] // 存储所有的订阅者 (观察者)
  }
}
```

```

}

// 添加订阅者
add(watcher) {
    this.subarr.push(watcher)
}

// 通知所有订阅者，调用它们的 update 方法
notify(message) {
    this.subarr.forEach(item => item.update(message))
}

// 移除订阅者
remove(watcher) {
    this.subarr = this.subarr.filter(item => item !== watcher)
}
}

// 定义一个订阅者 (watcher)
class Watcher {
    constructor(cb) {
        this.cb = cb // 订阅者的回调函数
    }

    // 当发布者通知时，执行订阅者的回调
    update() {
        this.cb()
    }
}

// 创建订阅者 w1
const w1 = new Watcher(() => {
    console.log("我是订阅者1")
})

// 创建订阅者 w2
const w2 = new Watcher(() => {
    console.log("我是订阅者2")
})

// 创建发布者 dp (订阅中心)
const dp = new Dep()

// 添加订阅者 w1 和 w2
dp.add(w1)
dp.add(w2)

// 触发通知，调用所有订阅者的 update 方法
dp.notify('')

//输出
//我是订阅者1
//我是订阅者2

```

执行流程

1. 创建 `Dep` 实例 `dp`（作为发布者，维护所有的订阅者）。
2. 创建两个订阅者 `w1` 和 `w2`，它们的 `update` 方法会在 `notify` 时执行。
3. 将 `w1` 和 `w2` 添加到 `dp` 的 `subarr` 数组，表示它们订阅了 `dp` 的消息。
4. 执行 `dp.notify()`，`Dep` 遍历 `subarr` 并调用每个订阅者的 `update()` 方法：
 - 触发 `w1.update()`，输出 "我是订阅者1"。
 - 触发 `w2.update()`，输出 "我是订阅者2"。

19、手写懒加载

只有进入视口的图片才会发起请求

```
// 获取所有的 img 元素
let images = document.querySelectorAll('img')

function lazyload() {
  // 获取当前滚动位置
  let scrollTop = document.documentElement.scrollTop
  // 获取窗口的可视高度
  let winHeight = window.innerHeight

  // 遍历所有图片
  for (let i = 0; i < images.length; i++) {
    // 判断图片是否进入可视区域
    if (images[i].offsetTop < scrollTop + winHeight) {
      // 将 data-src 中的真实图片路径赋值给 src，加载图片
      images[i].src = images[i].getAttribute('data-src')
    }
  }
}

// 监听滚动事件，每次滚动时触发 lazyload
window.onscroll = lazyload
```

20、手写深拷贝(处理循环引用)

`hasOwnProperty` 属性是否为对象的自有属性

```
function deepClone(obj) {
  let cache = new WeakMap(); // 创建 WeakMap 存储已拷贝的对象，防止循环引用

  function _deepClone(obj) {
    // 1. 处理基本数据类型和 null
    if (obj === null || typeof obj !== 'object') {
      return obj; // 如果是基本类型 (string, number, boolean, null, undefined, symbol)，直接返回
    }

    // 2. 处理循环引用
    if (cache.has(obj)) {
      return cache.get(obj); // 如果已经拷贝过这个对象，直接返回缓存的拷贝对象
    }
  }
}
```

```

// 3. 创建新的对象或数组
let result = Array.isArray(obj) ? [] : {}; // 通过 Array.isArray 判断是否为数组
cache.set(obj, result); // 先把当前对象存入 WeakMap, 防止后续循环引用

// 4. 递归拷贝对象的属性
for (let key in obj) {
  if (obj.hasOwnProperty(key)) { // 只拷贝对象自身的属性, 避免继承属性
    result[key] = _deepClone(obj[key]); // 递归拷贝子对象
  }
}

return result; // 返回拷贝后的对象
}

return _deepClone(obj); // 调用内部递归函数
}
const clonedObj = {
  name: "Alice", // 基本数据类型直接拷贝
  age: 25, // 基本数据类型直接拷贝
  details: { // 进入递归拷贝 details
    hobbies: ["reading", "traveling"], // 进入递归拷贝 hobbies
    address: { city: "Beijing", zip: 100000 } // 进入递归拷贝 address
  }
};

```

21、async await原理

```

let count = 0;
function request (url) {
  return new Promise(resolve => {
    setTimeout(() => {
      resolve(url + count++); // 每次调用, 返回 url 和一个递增的 count 值
    }, 500); // 模拟延迟 500 毫秒
  });
}

function* generate () {
  const res = yield request('1111') // 第一次调用 yield, 执行异步请求
  const res2 = yield request(res) // 第二次调用 yield, 执行另一个异步请求
  const res3 = yield request(res2) // 第三次调用 yield, 执行最后一个异步请求
  console.log(res3) // 输出最终结果
}

function run () {
  // 拿到生成器句柄
  const g = generate()

  function exec (params) {
    const { value, done } = g.next(params) // 每次调用 next, 生成器函数向前执行
    if (!done) {
      value.then(res => exec(res)) // 如果生成器函数没有结束, 等待 Promise 完成再继续执行
    }
  }

  exec() // 启动执行
}

```

```
run(); // 输出 1111012
```

22、手写LRU

LRUCache 的时间复杂度最大情况下依然是 $O(1)$

```
class LRUCache {
  constructor(length) {
    this.length = length; // 缓存的最大容量
    this.data = new Map(); // 使用 Map 来存储缓存数据，Map 保证插入顺序
  }

  // 存储数据，通过键值对的方式
  set(key, value) {
    const data = this.data;

    // 如果缓存中已经存在该键，先删除该键以更新顺序
    if (data.has(key)) {
      data.delete(key);
    }

    // 插入或更新缓存中的键值对
    data.set(key, value);

    // 如果缓存超出了最大容量，则需要删除最久未使用的数据
    if (data.size > this.length) {
      // 获取 Map 中的第一项，即最久未使用的数据
      const delkey = data.keys().next().value;
      // 删除最久未使用的缓存项
      data.delete(delkey);
    }
  }

  // 获取数据
  get(key) {
    const data = this.data;

    // 如果缓存中没有该键，返回 null
    if (!data.has(key)) {
      return null;
    }

    // 获取键对应的值
    const value = data.get(key);

    // 删除该键，将其标记为最新使用
    data.delete(key);
    // 重新插入该键值对，保证它成为最新使用的数据
    data.set(key, value);

    return value; // 返回获取到的值
  }
}

// 创建一个最大容量为 5 的 LRU 缓存实例
const lruCache = new LRUCache(5);
```

23、手写懒加载

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>

<body>
  <div class="container">
    
    
    
    
    
  </div>
  <script>
    let imgs = document.querySelector('img')
    function lazy () {
      let scrollTop = document.body.scrollTop ||
document.documentElement.scrollTop
      let winHeight = window.innerHeight
      for (let i = 0; i < imgs.length; i++) {
        if (imgs[i].offsetTop < scrollTop + winHeight) {
          imgs[i].src = imgs[i].getAttribute('data-src')
        }
      }
    }
    window.onscroll = lazy
  </script>
</body>

</html>
```

特性	懒加载 (Lazy Load)	虚拟列表 (Virtual List)
目的	延迟加载资源，减少初始加载时间。	渲染可视区域内的内容，减少 DOM 元素数量。
加载方式	只有在用户滚动或交互时，才加载资源。	只渲染当前可视区域内的元素，其他元素不在 DOM 中。
主要用途	主要用于图片、视频等资源的加载，或分页式数据加载。	主要用于大数据量的长列表或表格展示，提高渲染性能。
适用场景	图片、视频懒加载，数据懒加载等。	长列表或表格展示，特别是大量数据的动态渲染。
用户体验	可以减少页面的初始加载时间，但可能会有延迟加载的体验问题。	可以减少内存和渲染压力，提升渲染效率，避免因大量 DOM 元素造成卡顿。
实现复杂度	相对简单，主要通过监听滚动事件来加载资源。	实现较为复杂，需要计算和管理可视区域及动态加载/卸载 DOM 元素。

总结：

- **懒加载** 主要关注于延迟加载资源，尤其是图片、视频等内容，以减少初始加载的压力。
- **虚拟列表** 主要用于处理长列表或大量数据的高效渲染，确保只有可见区域的元素被渲染。

24、手写hardMan

实现一个 HardMan:

HardMan("jack") 输出:

I am jack

HardMan("jack").rest(10).learn("computer") 输出

I am jack

//等待10秒

Start learning after 10 seconds

Learning computer

HardMan("jack").restFirst(5).learn("chinese") 输出

//等待5秒

Start learning after 5 seconds

I am jack

Learning chinese

```
<!DOCTYPE html>
<html lang="en">
```

```

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>

<body>
  <script>
    // 定义一个构造函数 `hardMan`
    function hardMan(str) {
      this.queue = [] // 初始化任务队列
      this.name = str // 设置 `name` 属性为传入的名字
      this.queue.push(() => { // 初始化队列中添加第一个任务：打印名字
        console.log(this.name); // 输出名字
        this.next(); // 执行下一个任务
      });
    }

    // `rest` 方法，等待指定时间后执行任务
    hardMan.prototype.rest = function (wait) {
      const func = () => {
        setTimeout(() => { // 设置延时操作
          console.log(`Start learning after ${wait} seconds`) // 打印等待后开始学习
            的信息
          this.next(); // 执行下一个任务
        }, wait * 1000) // 延时 `wait` 秒
      }
      this.queue.push(func) // 将 `func` 任务加入队列
      return this // 返回当前实例，实现链式调用
    }

    // `restFirst` 方法，将延时任务加入队列的最前面
    hardMan.prototype.restFirst = function (wait) {
      const func = () => {
        setTimeout(() => {
          console.log(`Start learning after ${wait} seconds`)
          this.next();
        }, wait * 1000)
      }
      this.queue.unshift(func) // 将 `func` 任务加入队列的最前面
      return this // 返回当前实例，实现链式调用
    }

    // `learn` 方法，添加学习任务到队列
    hardMan.prototype.learn = function (str) {
      const func = () => {
        console.log(`Learning ${str}`) // 打印学习内容
      }
      this.queue.push(func) // 将 `func` 任务加入队列
      this.next() // 执行下一个任务
    }

    // `next` 方法，用来执行下一个任务
    hardMan.prototype.next = function () {
      if (this.queue.length === 0) return // 如果队列为空，直接返回
      const func = this.queue.shift() // 获取队列中的第一个任务
      func() // 执行该任务
    }
  </script>

```

```

// `HardMan` 工厂函数，创建一个新的 `hardMan` 实例
const HardMan = (name) => {
  return new hardMan("jack"); // 使用 "jack" 作为默认名字
}

// 调用 `HardMan` 创建一个实例，并依次调用 `rest` 和 `learn` 方法
HardMan("jack").rest(10).learn("computer");
// 你也可以取消注释并尝试使用 `restFirst` 方法
// HardMan("jack").restFirst(5).learn("chinese")
</script>
</body>

</html>

```

代码流程：

1. **hardMan 构造函数**：初始化一个队列 `queue`，并在队列中加入一个打印名字的任务。
2. **rest 方法**：向队列中加入一个延迟指定时间后执行的任务，实现链式调用。
3. **restFirst 方法**：与 `rest` 类似，但将任务加入队列的最前面。
4. **learn 方法**：向队列中加入学习任务，并开始执行队列中的任务。
5. **next 方法**：从队列中取出并执行下一个任务，直到队列为空。
6. **HardMan 工厂函数**：创建并返回一个新的 `hardMan` 实例。

通过这些方法，代码实现了一个任务队列，并允许以链式调用的方式添加不同的任务，并按顺序执行它们。

25、手写vue-router

```

let Vue

// VueRouter 类，模拟 Vue 的路由功能
class VueRouter {
  // 构造函数，接收路由配置 options
  constructor(options) {
    let initPath = '/' // 初始化路由路径为根路径
    // 使用 Vue 的响应式 API 实现路由的响应式数据绑定
    Vue.util.defineReactive(this, 'current', initPath)

    // 设置用户传入的路由规则
    this.routes = options.routes
    // 设置路由模式，默认为 'hash'
    this.mode = options.mode || 'hash'
    this.init() // 初始化，监听路由变化
  }

  // 初始化方法，根据路由模式监听路由的变化
  init() {
    if (this.mode === 'hash') {
      console.log(location.hash);
      // 页面加载时，获取当前 hash 路径
      window.addEventListener('load', () => {
        this.current = location.hash.slice(1) // 路径去掉 # 符号
      })
      // 监听 hashchange 事件，当路由发生变化时更新 current
      window.addEventListener('hashchange', () => {

```

```

        this.current = location.hash.slice(1)
      })
    }
  }
}

// VueRouter 插件的安装函数
VueRouter.install = function (_Vue) {
  Vue = _Vue

  // 在 vue 的生命周期钩子 beforeCreate 中，给组件实例添加 $router 属性
  Vue.mixin({
    beforeCreate() {
      if (this.$options.router) { // 根组件中传入的 router
        Vue.prototype.$router = this.$options.router // 将 $router 添加到 Vue 原型
        上
      }
    }
  })

  // 注册全局组件 router-link，用于页面跳转
  Vue.component('router-link', {
    props: {
      to: { // to 属性，表示跳转目标的路径
        type: String,
        require: true
      }
    },
    render(h) {
      return h('a', {
        attrs: { // 设置 a 标签的 href 属性，点击后会触发路由跳转
          href: "#" + this.to
        }
      }, this.$slots.default) // 默认插槽内容
    }
  })

  // 注册全局组件 router-view，用于显示当前路由对应的组件
  Vue.component('router-view', {
    render(h) {
      // 获取当前路由地址和路由配置
      let current = this.$router.current
      let routes = this.$router.routes

      // 查找与当前路径匹配的路由配置
      let com = routes.find(item => item.path === current)

      // 渲染匹配到的组件
      return h(com ? com.component : null) // 如果没有找到匹配的路由组件则返回 null
    }
  })
}

export default VueRouter

```

26、设计模式

1) 单例模式

一个构造函数一生只有一个实例。不管new多少次都是一个实例

应用：自定义弹出层

div默认隐藏 在一定条件下显示

单例模式核心代码：

```
let instance = null

function singleron(){
  if(!instance) instance = 实例对象
  return instance
}
```

```
const Person = (function () {
  function Person () {
    this.name = 'Jack'
  }
  Person.prototype.sayHi = function () {
    console.log('hello');
  }

  let instance = null

  return function () {
    if (!instance) instance = new Person()

    return instance
  }
})();

//再来创建
const p1 = Person()
const p2 = Person()

console.log(p1, p2)
console.log(p1 == p2); //true
```

2)观察者模式

例子：监控

监控一个对象的状态 一旦状态发生变化 马上触发技能

需要两个构造函数实现

1、创建被观察者

- 属性，自己的状态
- 队列，记录都有谁观察自己，数组[]
- 方法，能设置自己的状态，当我改变时，触发这个方法改变状态
- 方法，添加观察者
- 方法，删除观察者

2、创建观察者

- 需要一个身份证明
- 需要一个技能

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>

<body>
  <script>
    class Observer {
      constructor(name, fn = () => { }) {
        this.name = name
        this.fn = fn
      }
    }

    const bzs = new Observer("班主任", (state) => {
      console.log("因为" + state + "把你爸找来");
    })
    const xz = new Observer("校长", (state) => {
      console.log("因为" + state + "骂你的班主任");
    })

    console.log(bzs, xz);

    //被观察者的构造函数
    class Subject {
      constructor(state) {
        //记录自己的状态
        this.state = state
        //数组用来保存观察着我的人
        this.observers = []
      }
      //设置自己的状态
      setState (val) {
        this.state = val

        this.observers.forEach(item => {
          item.fn(this.state)
        })
      }

      addObserver (obs) {
        //判断观察者已经存在就不再添加
        let res = this.observers.indexOf(obs)
        if (res !== -1) return
        this.observers.push(obs)
      }

      delObserver (obs) {
        this.observers = this.observers.filter(item => item !== obs)
      }
    }
  </script>
</body>
</html>
```

```

    }
    xiaoming = new Subject("学习")

    xiaoming.addObserver(bzr)
    xiaoming.addObserver(xz)
    xiaoming.addObserver(xz)

    console.log(xiaoming);
    xiaoming.setState('玩耍')
    xiaoming.delObserver(xz)
    xiaoming.setState('爬树')
  </script>
</body>
</html>

```

3)策略模式

一个问题匹配多个解决方案，不一定用到哪一个

而且有可能随时继续添加多个方案

例子购物车结算

我们有很多折扣计算的方式

- 满100-10
- 满200-25
- 8折
- 7折

把多种方案以闭包的形式保存下来

- 对外留一个接口
- 可以增加和减少

```

<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>

<body>
  <script>

    const calPrice = (function () {
      const sale = {
        '100_10': function (price) { return price -= 10 },
        '200_25': function (price) { return price -= 25 },
        '80%': function (price) { return price *= 0.8 }
      }
      //被return 出去的函数 才是calPrice本体
      function calPrice (price, type) {

        if (!sale[type]) return '没有这个折扣'
      }
    })()
  </script>

```

```
        return sale[type](price)
    }

    //把函数当成对象往其中添加成员
    calPrice.add = function (type, fn) {
        //专门用来添加折扣
        //判断这个折扣是不是存在
        if (sale[type]) return "折扣已存在"

        sale[type] = fn
    }

    calPrice.del = function (type) {
        delete this[type]
    }

    return calPrice
})();

//将来使用的时候
const res = calPrice(320, '100_10')
calPrice.add('70%', function (price) {
    return price *= 0.7
})
console.log(res);
console.log(calPrice(320, '70%'));
</script>
</body>

</html>
```